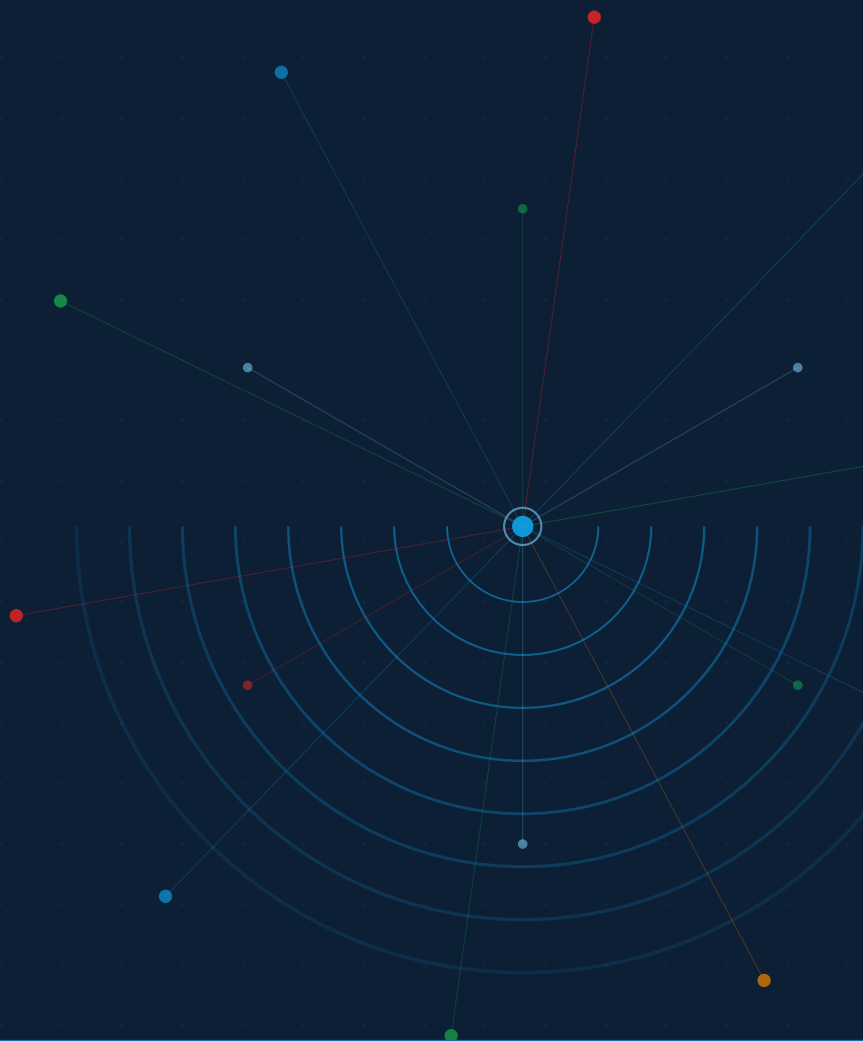


JA4proxy

Enterprise TLS Security Proxy

Product Overview & Buying Guide

For Security Buyers and Technical Evaluators



Contents

<i>The Problem: Bots Don't Look Like Bots Anymore</i>	3
<i>The JA4 Insight</i>	3
<i>Architecture: The Ultra-Lite Go Engine</i>	3
<i>High-Performance Observability</i>	4
<i>Ready for Enterprise</i>	4

The Problem: Bots Don't Look Like Bots Anymore

A CREDENTIAL-STUFFING CAMPAIGN LOOKS IDENTICAL TO HUMAN LOGIN TRAFFIC at the HTTP layer. The requests are syntactically valid. The user-agent strings are plausible. Your CDN's bot-score misses it because the attacker rotates IP addresses faster than reputation feeds update.

1

Traditional TLS-aware defences — SSL inspection and decryption — introduce significant latency and create a massive compliance exposure: PCI-DSS, HIPAA, and GDPR all restrict access to decrypted traffic. JA4proxy solves this by enforcing security *before* decryption, operating directly on the ClientHello handshake.

¹ The Verizon DBIR consistently shows that credential abuse accounts for more than 40% of breaches. Most credential-stuffing toolkits ship with TLS stacks that produce distinctive, stable fingerprints — yet traditional defences never inspect the handshake.

The JA4 Insight

BEFORE A SINGLE BYTE OF ENCRYPTED DATA IS EXCHANGED, every TLS client announces its capabilities in cleartext. This announcement — the ClientHello — contains the TLS version, cipher suites, and extensions (SNI, ALPN).

Because this information reflects the client's TLS stack, it is remarkably stable. Sliver 1.5.x produces the same ClientHello whether it is running from Frankfurt or Tokyo. Upgrading the *payload* has no effect on the fingerprint.

2

Client	JA4 Fingerprint (example)	Disposition
Chrome 125 (Win)	t13d1516h2_8daaf6152771_02713d6af86	ALLOW
Firefox 126	t13d1915h2_cd87aab0d655_5d1a4de8bd1	ALLOW
Sliver C2	t13d2012_c1e4e3a9b26a_000000000000	BLOCK
Cobalt Strike	t13d1112_b4f47bba57b5_9b5e0e9c9d74	BLOCK

² **Elite Performance:** The JA4proxy Go core processes these fingerprints in less than 27 nanoseconds, enabling theoretical throughput of 6 million connections per second per core. C2 frameworks typically do not.

Architecture: The Ultra-Lite Go Engine

JA4PROXY V2.0.0 IS A PURE GO-NATIVE IMPLEMENTATION designed for extreme resource efficiency and sub-microsecond latency. The system is distributed as exactly two binaries:

- **JA4proxypd (The Engine):** A stateless, high-concurrency traffic daemon that handles TLS-passthrough at scale.
- **JA4proxyp (The Toolset):** A unified operational CLI for management, real-time simulation, and integrated high-speed benchmarking.

High-Performance Observability

VISIBILITY SHOULD NOT COME AT THE COST OF PERFORMANCE. JA4proxy includes integrated real-time metrics for Prometheus and Grafana, providing granular visibility into the security waterfall.

Observation	Latency (p99)	CPU Impact
Bypassed (Fast Path)	272 ns	< 0.1%
Scored (Standard Path)	2.3 μs	< 2%
Adversarial (JA4X)	6.0 μs	< 5%

³ JA4proxypd has a memory footprint of less than **12 MB** and starts in under **50 ms**, making it ideal for containerised auto-scaling environments.

Table 2: Measured processing overhead per connection on reference server hardware (Intel i9-9900K).

Ready for Enterprise

JA4PROXY V2.0.0 IS PRODUCTION-CERTIFIED for high-traffic DMZ deployments. The system includes integrated cluster synchronization, Zero-Trust Redis hardening (TLS), and PROXY protocol v2 support with TLV-based trust verification.

Source code and technical evaluators guide:
<https://github.com/seanpor/JA4proxy>
JA4 Specification: <https://github.com/FoxIO-LLC/ja4>

What JA4proxy does

JA4proxy intercepts the TLS ClientHello — which is always transmitted in plaintext, before encryption begins. It computes a JA4 fingerprint from the cipher suites, extensions, ALPN, and elliptic curves advertised by each client, then routes the connection through a multi-signal scoring pipeline to decide: *allow, tarpit, block, or ban*. Allowed connections are forwarded byte-for-byte unchanged. The proxy never decrypts, never holds your private keys, and is transparent to both client and backend.

Key facts

- 0% false positive rate on browser traffic — by design
- 94–99% of malicious traffic blocked in testing
- Default dial = 0: monitor mode, never blocks on first deploy
- All bans carry a 5-minute TTL — false positives self-heal
- Never decrypts: no key custody, no SSL inspection compliance risk
- Scales horizontally; Redis synchronises bans across all instances

Resources

Source code & issue tracker: github.com/seanpor/JA4proxy

JA4 fingerprint specification: github.com/FoxIO-LLC/ja4

Documentation index: `docs/INDEX.md` in the repository